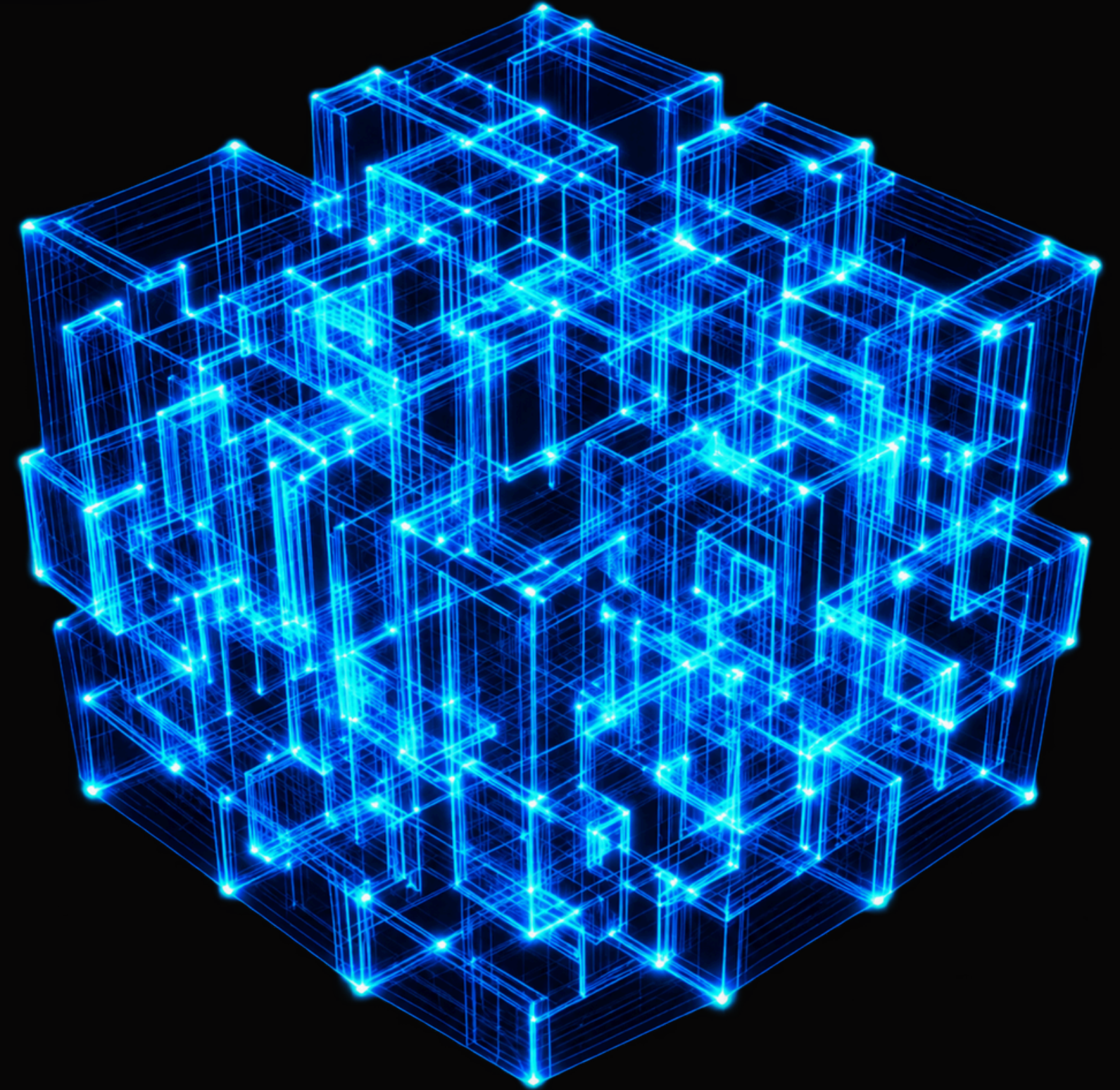


3D GRAPHIC RENDERING USING MATRIX CHAIN MULTIPLICATION



Section – 7

Team Members:

2520030231 – K. Girish

2520030450 – P. Venkat

2520030307 – N.V.Karthikeya

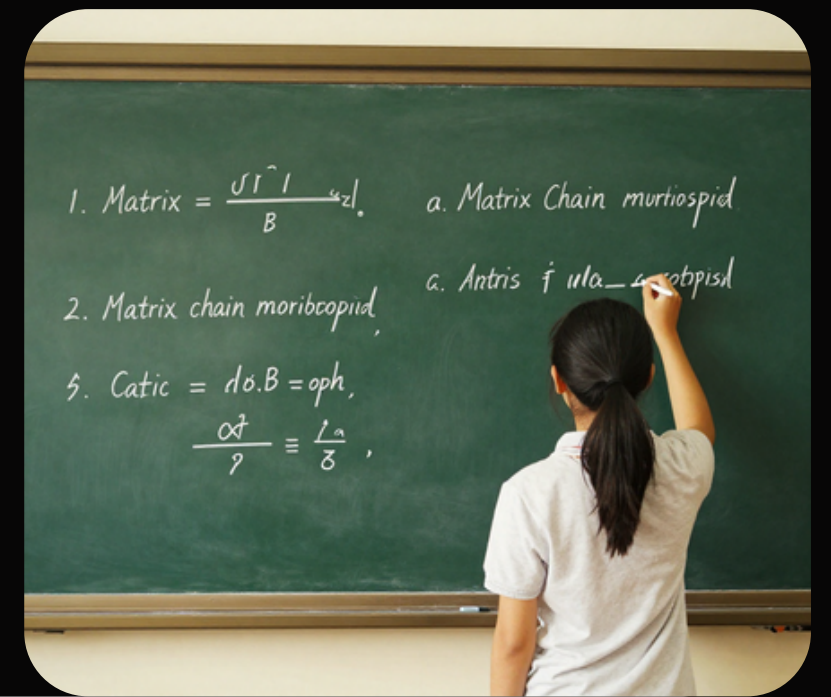
Team : 09

Abstract

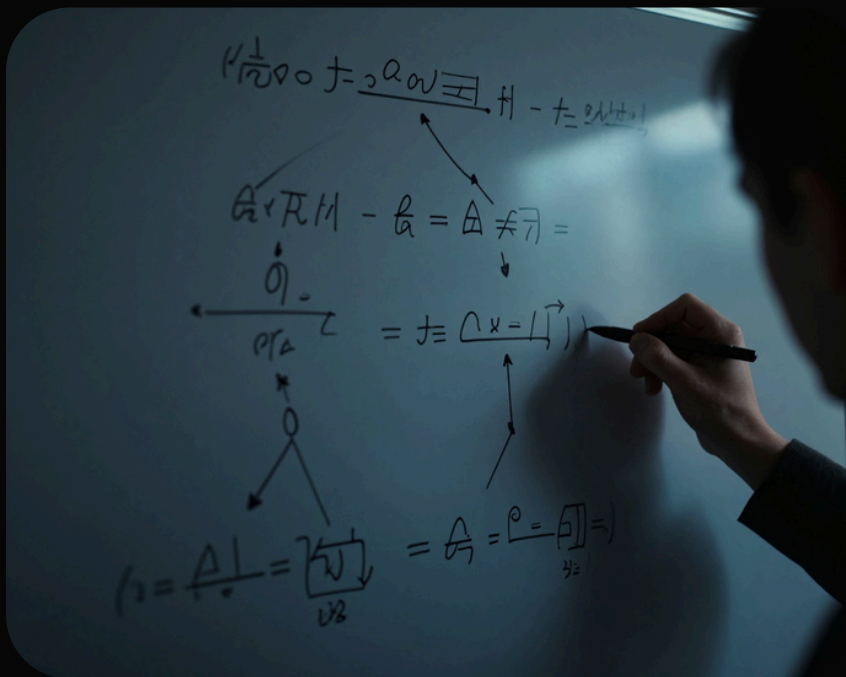
- **3D Graphic Rendering Using Matrix Chain Multiplication is a project that focuses on optimizing matrix operations involved in 3D graphics. Various transformation matrices such as translation, rotation, scaling, and projection are required to render and transform 3D objects. Multiplying these matrices in an inefficient order can increase computational cost. This project uses the Matrix Chain Multiplication algorithm with Dynamic Programming to determine the optimal order of matrix multiplication while minimizing the number of scalar multiplications. The system calculates the minimum computational cost and displays the optimal parenthesization of the matrix chain. This approach helps reduce unnecessary calculations, improves computational efficiency, and demonstrates how Data Structures and Algorithms can be applied to real-world computer graphics and rendering applications.**



Problem Statement

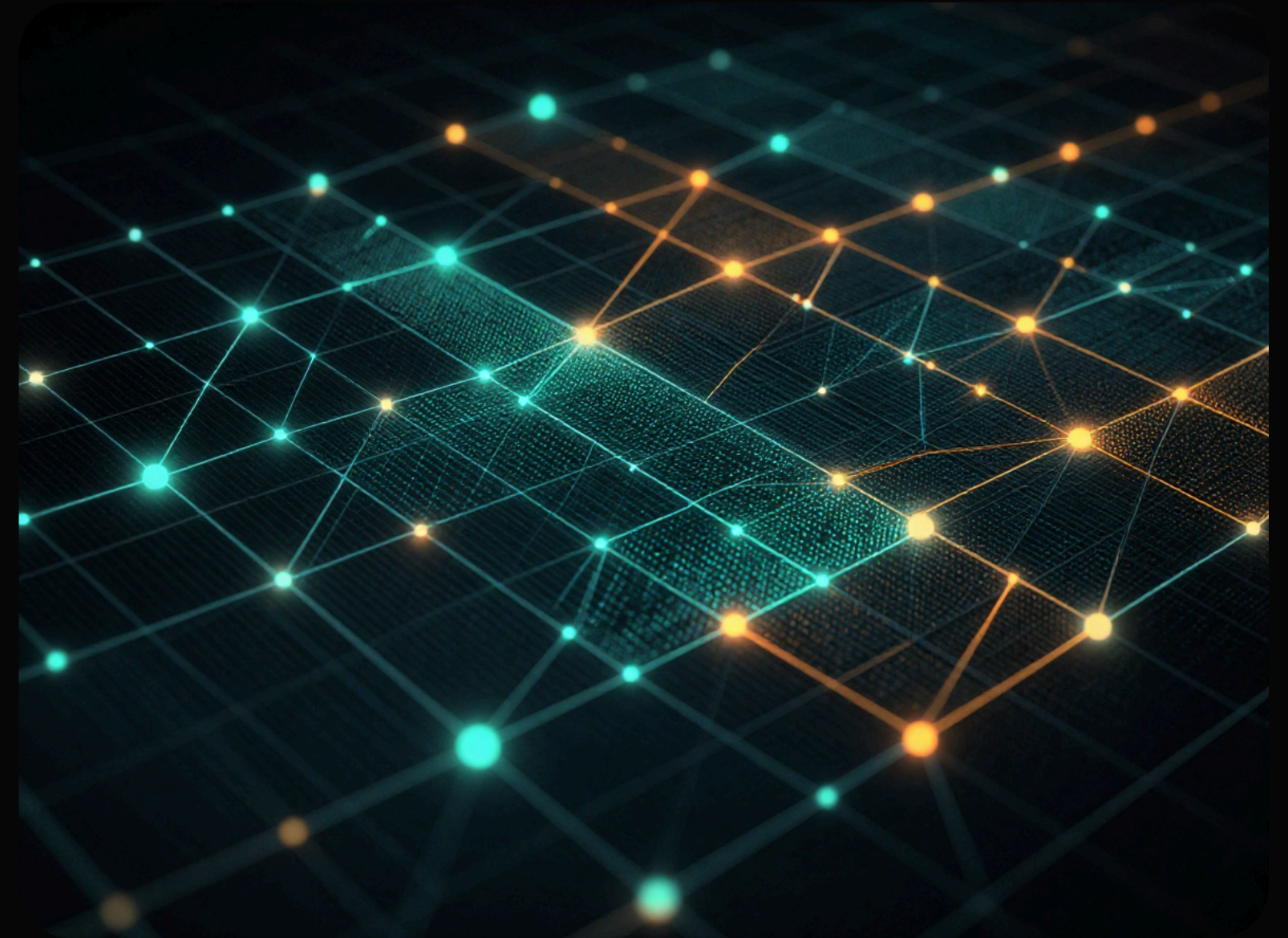


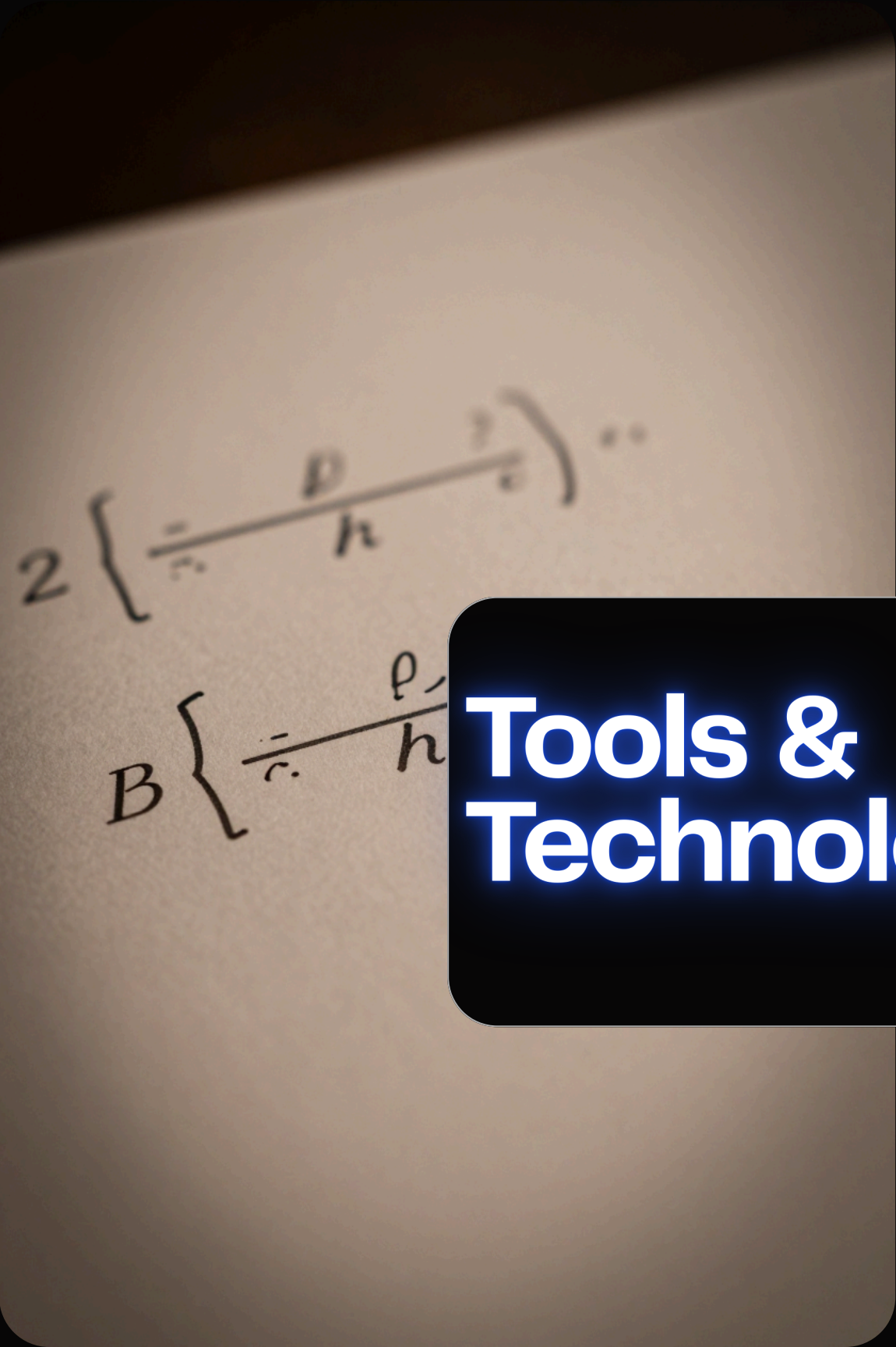
- 3D rendering requires multiple transformation matrices.
- These matrices are used for rotation, scaling, translation, and projection.
- Multiplying matrices in an inefficient order increases the number of computations.
- Different multiplication orders can produce the same final result but have different computational costs.
- The project aims to find the optimal multiplication order for the given matrices.
- Matrix Chain Multiplication using Dynamic Programming is used to minimize the number of scalar multiplications.
- This helps improve processing efficiency and rendering performance.
- The project demonstrates the practical use of DSA in computer graphics.



Proposed Functionalities

- 3D rendering requires multiple transformation matrices.
- These matrices are used for rotation, scaling, translation, and projection. ◀
- Multiplying matrices in an inefficient order increases the number of computations.
- Different multiplication orders can produce the same final result but have different computational costs.
- The project aims to find the optimal multiplication order for the given matrices.
- Matrix Chain Multiplication using Dynamic Programming is used to minimize the number of scalar multiplications.
- This helps improve processing efficiency and rendering performance.
- The project demonstrates the practical use of DSA in computer graphics.





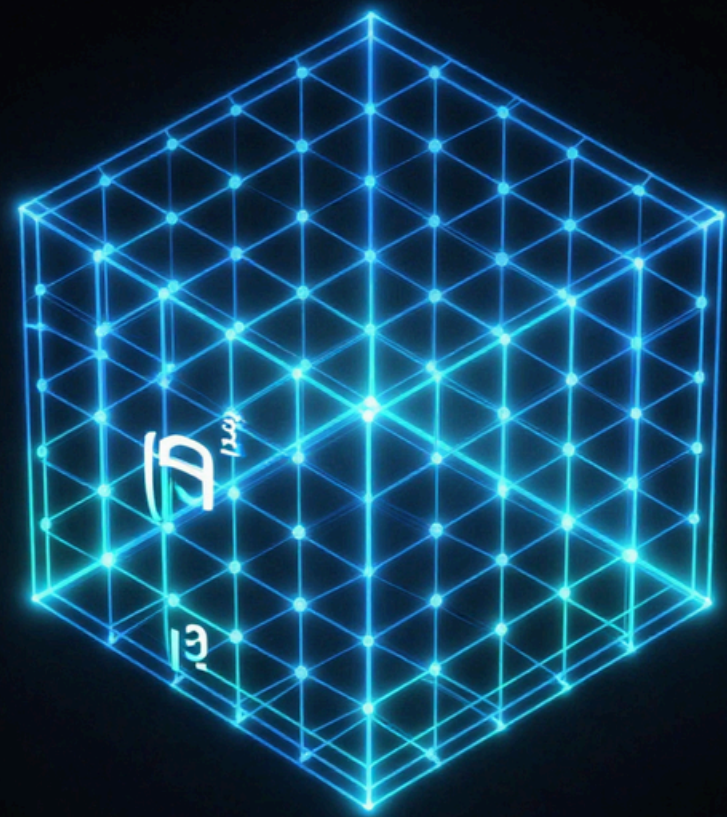
Handwritten formulas on a notebook page:

$$2 \left\{ \frac{D}{n} \right\} \dots$$
$$B \left\{ \frac{D}{n} \right\}$$

Tools & Technologies

- **Programming Language: Java**
- **Algorithm: Interval Dynamic Programming**
 - **Data Structures: 2D Arrays**
 - **DP Cost Table**
 - **Split Table**
 - **Development Environment: VS Code / IntelliJ IDEA**
- **Version Control: Git & GitHub**
- **Complexity:**
 - **Time: $O(n^3)$**
 - **Space: $O(n^2)$**

Expected Output



INTERVAL DYNAMIC PROGRAMMING

MATRIX CHAIN OPTIMIZER

Visualize optimal matrix multiplication using dynamic programming.

01 MATRIX CONFIGURATION

Choose the number of matrices and enter their dimensions.

NUMBER OF MATRICES

2

BUILD MATRICES

DIMENSIONS

N MATRICES = N + 1 DIMENSIONS

ROWS

10

D1

20

FINAL COLS

30

MATRIX CHAIN

A1 × A2

02 3D MATRIX VISUALIZATION

Watch the matrices move and merge during the optimization process.

A1

10×20

MCM

DP CORE

A2

20×30

READY

◀ PREV

▶ PLAY

NEXT ▶

🔄 RESET

03

OPTIMIZATION

Calculate the minimum scalar multiplication cost.

⚡ OPTIMIZE MATRIX CHAIN

04

OPTIMIZATION RESULTS

Comparison between optimized and naive multiplication.

OPTIMIZED COST

6,000

SCALAR MULTIPLICATIONS

NAIVE COST

6,000

LEFT → RIGHT

OPERATIONS SAVED

0

OPERATIONS

EFFICIENCY

0.00%

IMPROVEMENT

OPTIMAL PARENTHEZIZATION

(A1 × A2)

05

DP COST TABLE

Minimum multiplication cost for every matrix interval.

0

6,000

-

0

DP FORMULA $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + p[i-1] \times p[k] \times p[j])$

Thank You

